

The Future Unemployed

The NEXT WAVE IN TECHNOlogy

April 2, 2026

Gracie Mitchell, Graham McCabe, Reid Holloway
Panther Cloud Air | Panthercloudair@gmail.com

Table of Contents

Introduction **5**
- Design Consideration 6
- Assumptions and Dependencies 6
  - User Assumptions 6
  - System Architecture Assumptions 6
  - Data Assumptions 7
- Goals and Guidelines 7
  - System Goals 7
  - Consistency and Design Standards 8
  - Data Integrity Guidelines 8
- Development Methods 8
  - Prototype-Driven Development 8
  - Iterative Development Process 8
  - Component Implementation 8
  - Frontend and Backend Integration 9
  - Testing and Improvement 9

Architectural Design **9**
- Data and Control Flow 9
- Database 9
- Derived Program Structure 11

Interface Design **13**
- Assumptions 13
  - Employee 13
  - Admin 14
- Goals 15
  - Clarity 15
  - Efficiency 16
  - Visual Consistency 16
- Guidelines 16
  - Consistency 16
  - Hierarchy 16
  - Error Prevention and Feedback 17
- Development Methods 17
- Use Case 18

Procedural Design **18**
- Processing Narrative 18
- Interface Description 18
- Design Language Description 19
- Use Cases 19

Updated MS Project Reports - Schedule and Costs..... 22
Schedule.....22
Early Semester Meetings.....24

Table of Figures

Figure 1: Relational Database Model....8
Figure 2: System Diagram....10
Figure 3: Employee Design....11
Figure 4: Admin Design....12
Figure 5: Interface Design Diagram....16
Figure 6: Employee Book Flight Flow Chart....20

Table of Tables

Table 1: Database Table.....10
Table 2: Procedural Design Use Cases..... 21

Introduction

Panther Cloud Air is a prototype airline operations and simulation system designed for a startup airline seeking to evaluate the feasibility of its business model before full implementation. The system was created for a small group of investors who want to determine whether an airline focused on profitability, quality service, and reasonable ticket pricing can compete effectively in the current market. The prototype supports several major business functions, including user login by role, flight schedule viewing, customer booking, employee booking and passenger management, and administrative oversight of operations.

Paragraph on Project Description

The purpose of this document is to present the updated design of the Panther Cloud Air prototype and explain how the system has progressed since the earlier project plan. This document summarizes the major design decisions, identifies assumptions and constraints, records changes made during development, and provides supporting project management information. It also serves as a communication tool for the client by documenting the current state of the prototype, the remaining work, and the overall direction of the system as the team moves toward final implementation and demonstration.

Changes to Previous Plan

Several changes were made to the previous plan as development progressed. Initially, the project focused primarily on simulating airline business feasibility at a high level. The updated prototype now places greater emphasis on an integrated user experience through a web-based operations portal. In addition to the original concept of simulation, the system now includes a more developed customer booking workflow, employee-facing booking and passenger lookup tools, and an administrator dashboard that supports operational review and pricing analysis. The revised plan also places more attention on role-based access, usability, and visual presentation so the system can function both as a business simulation and as a prototype of an operational airline management interface. These changes improved the realism of the project and better

aligned the prototype with the client's goal of evaluating both logistics and profitability.

Design Consideration

Assumptions and Dependencies

Panther Cloud Air bases itself on several foundational assumptions in regards to its users, overall system environment, and data availability which are key to the backend and frontend.

User Assumptions

The system contains two main types of user:

- Employees
    - They interact with operational features such as flight booking, schedule viewing, and live status tracking.
- Administrators
    - They contain the same features as employees but are also given access to system-level data such as pricing, fleet performance, and simulation results.

The interface assumes users are familiar with web-based systems but maybe not with airline specific ones. For this reason, we combined simplicity and efficiency. Displaying the different tabs of the site at the top provides any user to swiftly switch tasks as well as being an easy-to-learn system for those who may be new.

System Architecture Assumptions

The design contains a multi-layered system architecture.

1. Database Layer
    a. These are our SQL files organized in a way where (see Table 1)
        i. Our relational data is displayed clearly (tables, schedules, routes)
        ii. Forms map directly to structured schema (Users are not given the opportunity to freely put in data)
2. Backend Layer
    a. This is meant for
        i. UI actions, like flight booking or searching, that must pull from backend functions

        ii. This interface must support:
            1. Asynchronous data loading
            2. Real-time updates for live status tab
    b. This is all done by using the files
        i. app.py
            1. Application logic
        ii. db.py
            1. Database connection and queries
3. Container (Docker)
    a. Using files like
        i. Dockerfile
        ii. docker-compose.yml
        iii. entrypoint.sh
    b. This ensures consistency that UI must not rely on environment-specific behavior. It supports scalability and testing across the entire system.

Data Assumptions

Our system assumes three main things with the data:

1. All airport and flight data is preloaded and valid.
    a. No flight calculations will be made but done beforehand.
2. Simulation data can be computed dynamically.
    a. To compute simulation data, randomization and delays will be added to already existing routes.

Our user interface focuses on selection over input. The research has already been done, to make the code simpler it will make as few calculations as needed. This means our data will be displayed in structured formats rather than raw text due to being mostly prematurely calculated.

Goals and Guidelines

System Goals

The goal of Panther Cloud Air's interface is to provide a clear and efficient platform for managing airline operations such as flight booking, scheduling, and administrative tasks. The system is designed to support multiple user roles. This is to ensure that employees and administrators can access tools relevant to their roles. One objective is to make workflows

efficient so every user can complete tasks quickly with minimal errors. It also aims to provide clear and consistent information to the user so they can easily understand and trust the system.

Consistency and Design Standards

Consistency in the system is maintained across the interface through consistent colors and fonts that are soothing to the eye. Similarly, every action behaves in predictable ways, allowing users to quickly learn how to interact with the system. This improves usability and makes future development easier by creating clear design patterns.

Data Integrity Guidelines

The system is designed to make sure that data stays accurate. User inputs are validated before submission to prevent errors, and information is displayed in a structured format to avoid misinterpretation by the user. Clear communication between the frontend and backend confirms data is updated correctly.

Development Methods

Prototype-Driven Development

Our development started with an HTML and CSS prototype that set the basis for our intended layout and user interface. It allowed our team to visualize the system early, understand its usability, and identify potential issues before fully implementing the databases.

Iterative Development Process

We developed our system in stages by constantly refining our system and getting feedback. Each iteration gave more and more insight on usability, layout, and interaction flow, allowing the design to evolve into a more polished and effective interface.

Component Implementation

The interface was built using reusable components such as navigation bars, forms, and tables. This ensures consistency across the system and simplifies updates, as changes only affect the tables and not the overall layout of the system.

Frontend and Backend Integration

The design contains backend functionality so a user's actions affect system functions. Forms, data displays, and interactions are structured to support dynamic data and constant communication with the database.

Architectural Design

Data and Control Flow

Our existing database consists of 14 different tables in 8 different categories that are used to run this simulation and website. The goal for this database was to keep it as simple as possible but making information easy to store and get when needed in the system. The system uses SQL queries to retrieve this information and using flask that's connected to docker we're able to retrieve that data to turn into something we can use for our front end website.

Database

Category	Table Name	Table Description
Aircraft	Aircraft	Individual aircraft registered in the fleet.
Aircraft	Aircraft_models	Stores specifications for each aircraft type
Airports	Airports	All operating airports.
Flight Distances	Flight_distances	Point to point distances in NM from each Airport.
Flights	Flights	Tracks each flight operated by Panther Cloud Air.
Flights	Turnaround_time	Tracks ground time breakdown for each flight.
Fuel	Flight_fuel_summary	Rolls up all phase logs into a single record per flight.
Fuel	Flight_fuel_logs	Tracks fuel burn for each phase of a flight.

Fuel	System_config	Stores admin-controlled simulation-wide settings.
Passenger	Bookings	Stores flight reservations for each passenger.
Passenger	Passengers	Stores personal information for each passenger.
Pricing	Ticket_prices	Calculated ticket price per flight and seat class.
Pricing	Pricing_rules	Stores base pricing per seat class
Users	Users	Stores all registered system users.

Table 1: Database Table

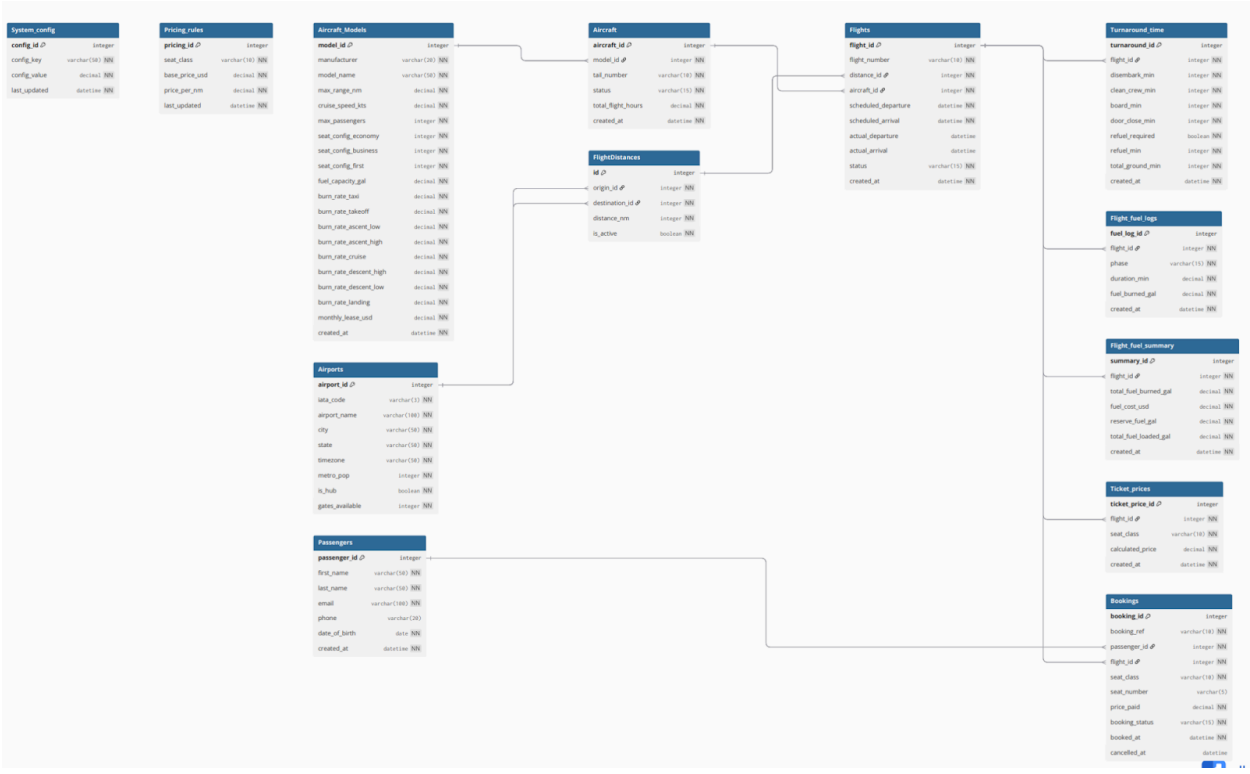


Figure 1: Relational Database Model

Derived Program Structure

The Panther Cloud Air system is organized into four hierarchical layers, each derived from the functional requirements of the simulation. The top-level system descends into the following major modules:

1. **Frontend**: The user-facing website built with HTML/CSS/React. It contains 5 primary view modules: the Login Module which handles role-based authentication routing for Passenger, and Admin users, the Schedule Module which displays filterable flight tables with live status, the Booking Module where users can book flights for customers, the Employee Module which is subdivided into Flight Search, Manage Bookings, Passenger Lookup, and Live Status panels, and the Admin Module which is subdivided into Overview, Route Profit, Fleet, Simulation, Pricing, and Users panels. Navigation between these views is managed by a client-side Each page will be served by Flask and rendered based on the authenticated user's role.
2. **Application Logic Layer**: This layer is handled by Flask, which serves as the controller between the frontend pages and the database. It contains route handlers that map HTTP requests to the correct page or query, a business logic module that computes ticket fares using the 30% load factor assumption and margin adjustments by route type, and an authentication module that enforces role-based access control across all pages.
3. **Data Access Layer:** This consists of SQL query modules organized around the eight database categories — Aircraft, Airports, Flight Distances, Flights, Fuel, Passenger, Pricing, and Users — each responsible for reads and writes to its respective tables.
4. **Infrastructure Layer:** This consists of the MariaDB instance running inside Docker, which physically stores all 14 tables and is accessed exclusively through the Flask data access layer.

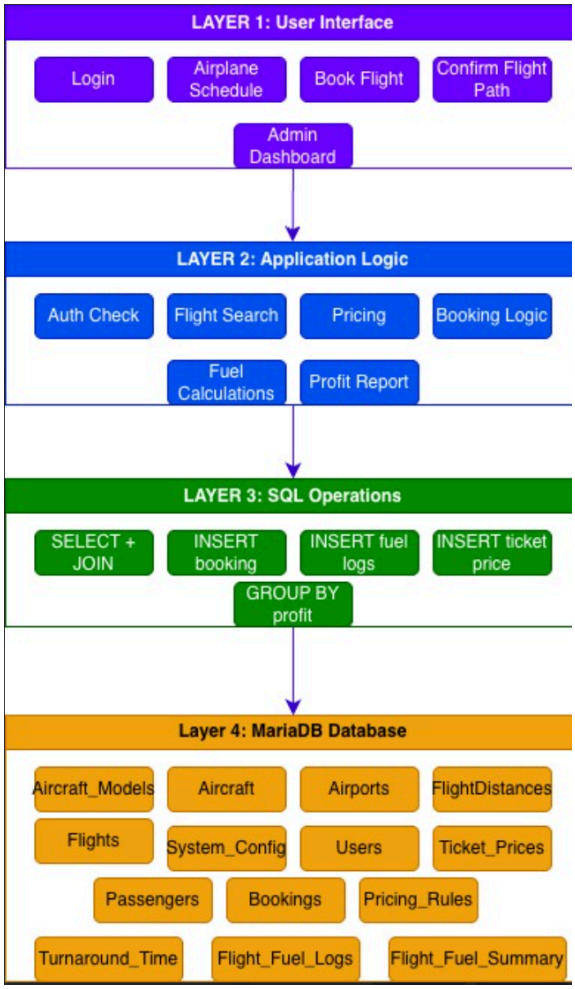


Figure 1: System Diagram

Interface Design

Assumptions

The users of this application will be one of two people

- Employees
- Administration

Employee

The application design is focused towards them. Everything the employee can see, the administration account can also see. There are no special

actions or tabs in which the employee only can see. This makes the design for them sleek and easy to understand. At the top, there are tabs for each section, allowing the user to see the broad variety of actions they are able to take.

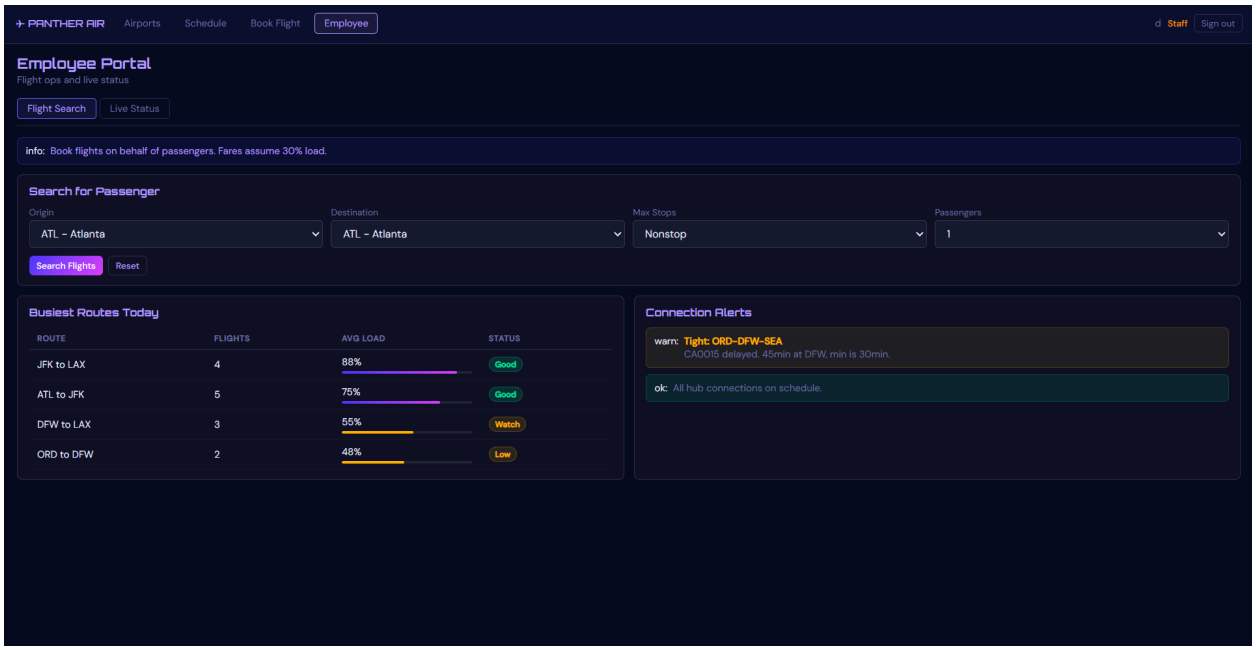


Figure 2: Employee Design

The employee is able to choose from

- Airports
    - Here the user is able to view all airports and their specifications. The user can get more familiar with the system and airline running to better understand what flights may or may not be as easy to book.
- Schedule
    - A live look at all flights through every airport consisting across the entire day. Provides an overview of the constant flight pattern of the airline.
- Book Flight
    - A user can choose where they want to book a flight to and from.
- Employee
    - Contains two more tabs
        - Flight Search
        - Live Status

- Both give a current flight status rather than an overall view of the scheduling. Provides a more active understanding of flights as they are occurring.

A new employee may learn more about the airline and they system with certain tabs while also containing tabs that long-term employees, who already know the system, may perform their tasks with ease.

Admin

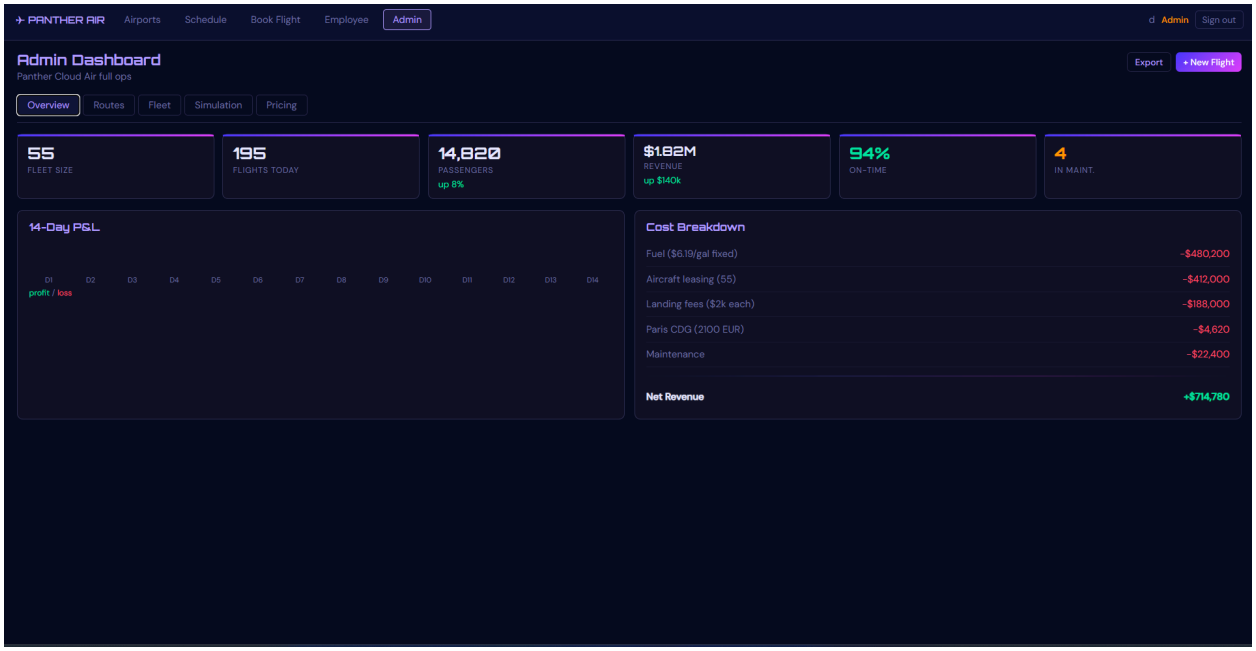


Figure 3: Admin Design

The administration is able to access all the same functions as the employee. If they are new or old, this design allows for and easy-to-understand layout of their permissions along with the site's intent. The exception for them is the admin dashboard which includes

- Admin overview
- Routes
    - Not just the schedule of flights but with specifics like revenue, flight capacity, and overall costs for the flight.
- Fleet
    - A view of every aircraft with its revenue and condition over the course of its existence within the company.
- Simulation

    - A 14-day simulation involving different variables to see the flight-routing's overall profitability even through the worst possibilities.
- Pricing
    - The ability to adjust profit per ticket to determine the financial benefit of the company.

These functions are not accessible to the employee due to the finances shown but are labeled accordingly to allow ease for admin to view the necessary details they need to see.

Goals

Clarity

The interface is intentionally designed to properly separate employee and administration roles. An employee can properly see where they may complete certain tasks. In the admin tab, administrators are provided a visual to all financial and important documentation that only they can see.

Instead of having tabs all over the screen in order to figure out where to go, the tabs are laid out at the top at all times giving whoever is using the site an easy to navigate system.

Efficiency

All users are able to book flights quickly and view flight schedules without confusion. The book flights tab contains a drop down menu with not just the IATA but city name as well. This provides convenient use for employees who may not know the code but the name of the city. If a passenger approaches them knowing where they would like to go but not the area code, the employee can efficiently find the proper location. A "swap" button is also present to find returning flights for the passenger if they would like a return flight as well.

Visual Consistency

A consistent color palette and layout is used for easier site comprehension. All important items are located in the same place providing a layout with little confusion. If an employee or administrator are in a time crunch and must maneuver around the site quickly, every feature is always accessible in the same location.

Guidelines

Consistency

We want all users to have a feel and get comfortable with the site. If a user is constantly needing to refresh themselves with the site and are left confused then it is not a productive site. The consistent tab placement and color palette gives the user a familiarity which allows for a more productive workflow.

Hierarchy

All clickable buttons are highlighted to pop out to the user. There is not a button that is hidden or hard to find. Items that are not able to be interacted with are important but if it is important it will be on the page. We laid out the buttons in this way so navigation to information is easy and action icons stick out. If there is a new user, they are not stuck wondering where they need to go but can easily traverse the site to accomplish whatever task required.

Error Prevention and Feedback

The user interface is designed for users to work efficiently while minimizing errors. The system avoids free-form inputs by the user; providing drop-down menus for airport selection, dates, and connections reduces confusion by not requiring the user to guess what inputs are allowed. They are shown what they can and cannot do to avoid redundant errors with airports or dates that may not exist.

There is also a workflow order to avoid tasks being done in an incorrect order. For example, if a user intends to book a flight, they are not allowed to input any personal information of the passenger until they have selected a flight.
When booking a flight, there is a confirmation message. This confirms the user is not in confusion if the flight was booked but that the information they input was correct. If it was not, the page will not continue but indicate that they must input the proper data to continue.

Development Methods

When designing, we started with a basic format containing all the basic functionalities such as overhead tabs, drop down menus, and role selections. Establishing a working model early allowed us to

- Evaluate usability and layout decisions before implementing backend data
- Identify design issues early
- Ensure the interface aligns with requirements

We used an iterative process that involved trial and error. As we fixed certain parts of the code, we continually tested its effects to all parts of the site to prevent long term errors in other areas.

The design is meant to integrate data through SQL in the backend. With pages such as “Book Flight” and “Schedule” having no use when there is no data in there. This is to set the framework of what we want the site to look like then implement the data rather than fixing the site based on the data.

Use Case

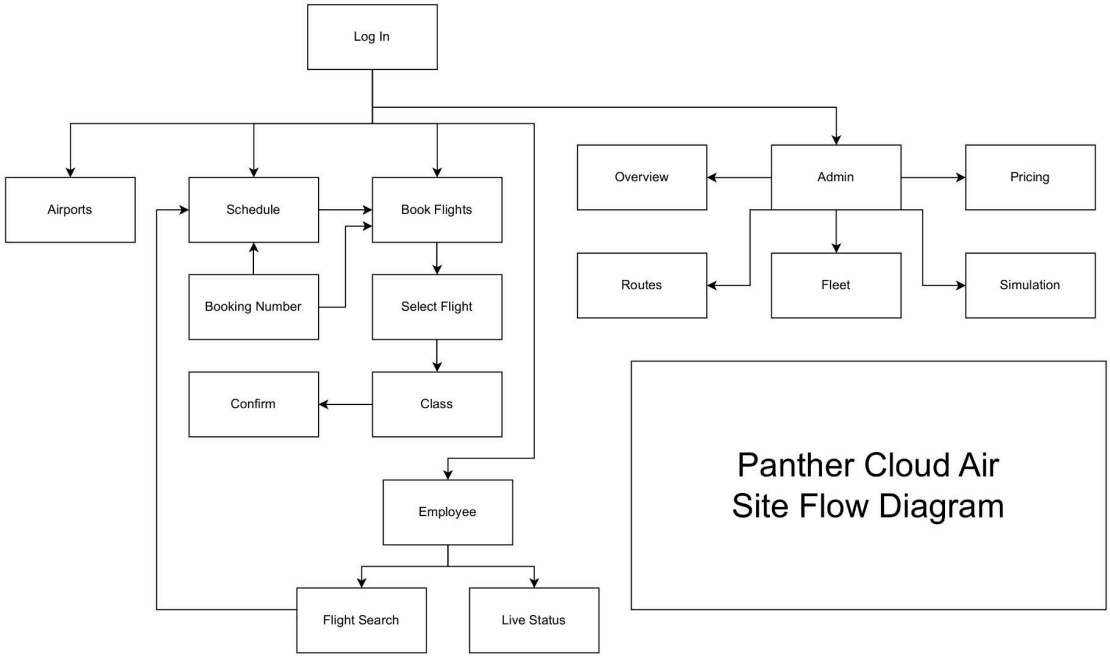


Figure 1: Interface Design Diagram

Procedural Design

Processing Narrative

The Panther Cloud Air system works by taking user input from the website and sending it to the Flask backend, which then runs the appropriate SQL query on our MariaDB database running in Docker. Once the query runs, the data gets sent back to the frontend and displayed to the user. For something like calculating a ticket price, Flask pulls the base pricing and fuel cost from the database and returns the final fare. The 14-day simulation runs one day at a time, where each day has a specific challenge applied to it, and all the results get saved to the database so they can be queried and reviewed afterward.

Interface Description

There are two main interfaces in our system. The first is the website itself, which is built with HTML and React and served through Flask. Depending

on whether you log in as an employee or an administrator, you will see different pages and options. Employees can search flights, manage bookings, look up passengers, and check live flight status. Administrators get all of that plus access to profitability reports, fleet management, pricing controls, and the simulation panel. The second interface is the Flask API that sits between the website and the database. All database reads and writes go through Flask, nothing in the browser talks to the database directly, which helps keep things secure and organized.

Design Language Description

Our system follows a layered architecture where each part of the stack has a specific job. The frontend is built with HTML and React and handles everything the user sees and interacts with. Flask acts as the middle layer, handling routing, business logic, and all communication with the database. MariaDB stores all of our data and runs inside a Docker container so the environment stays consistent no matter where it is deployed. The database is split into eight categories: Aircraft, Airports, Flight Distances, Flights, Fuel, Passenger, Pricing, and Users. Each category maps to one or more tables. We used foreign keys and status field constraints throughout the schema to make sure the data stays clean and consistent.

Use Cases

ID	Function Name	Actors	Description
UC.01	View Flight Schedule	Employee, Admin	User views a filterable table of all scheduled flights including status, load, and gate.
UC.02	Search Flights	Employee, Admin	User searches by origin, destination, and date to find available routes and estimated fares.
UC.03	Book Flight	Employee,	User enters

		Admin	customer info and the system creates a passenger record and confirmed booking.
UC.04	View Live Flight Status	Employee, Admin	User views real-time flight data including delays, gate assignments, and passenger counts.
UC.05	Configure Pricing	Admin	Admin views aircraft performance, maintenance status, and daily profit or loss per plane.
UC.06	Run Simulation	Admin	Admin can run simulation

Table 1: Procedural Design Use Cases

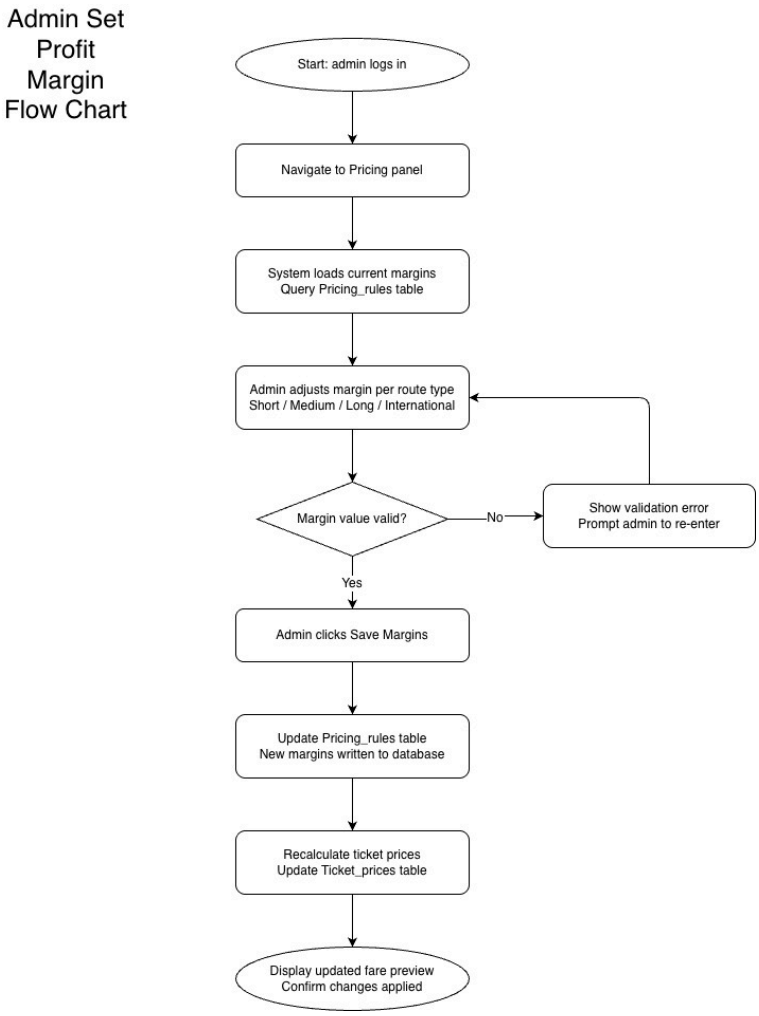


Figure 1: Admin Set Profit Margin Flow Chart

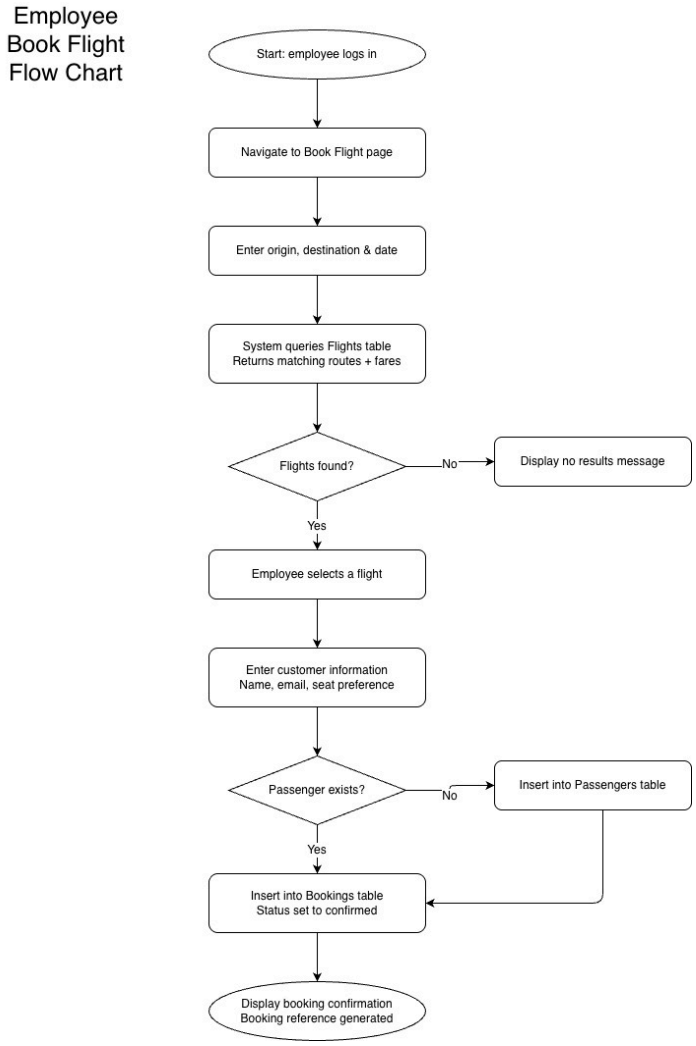


Figure 1: Employee Book Flight Flow Chart

Updated MS Project Reports - Schedule and Costs

Schedule

The updated project schedule reflects the team's transition from early planning and requirements gathering into interface development, functional prototyping, and project documentation. Early tasks focused on understanding client needs, defining the purpose of the airline simulation, and identifying the core functions needed in the prototype. The middle

phase of the project emphasized interface design and construction of the main system pages, including login, schedule display, booking workflow, employee tools, and administrative controls. The final phase centers on testing, documentation, presentation preparation, and refinement of system details for demonstration.

At this stage, major prototype components have been completed or substantially developed. The user interface structure is in place, and the project now demonstrates the primary workflow expected by the client. Remaining tasks are concentrated on final adjustments, validation of system behavior, completion of documentation, and preparation for the end-of-semester presentation. Overall, the project remains aligned with the semester timeline, although some tasks shifted slightly.

Costs

The updated cost report estimates the total development cost of the Panther Cloud Air prototype based on team labor hours. Since this project is a software prototype developed for demonstration purposes, the primary cost factor is the estimated labor required to design, build, test, and document the system.

For this project, the software development hourly rate of $60 per hour was used to estimate the total cost. This rate reflects a reasonable approximation of professional software development labor and provides a realistic estimate of the cost required to develop a similar system for a client.

The project development time was divided into several major phases, including planning, design, implementation, testing, and documentation. Requirements Planning: 20 hours

Project Meeting Notes

Throughout the semester, the Panther Cloud Air development team held regular meetings to plan, design, develop, and refine the airline simulation prototype. Meetings were held weekly to ensure steady progress, assign responsibilities, review completed tasks, and resolve any issues that arose during development. The following notes summarize the major phases of project meetings over the semester.

Early Semester Meetings

Focus: Understanding project goals and defining system requirements

During the early weeks of the semester, the team met to review the client's request and discuss the purpose of the Panther Cloud Air system. The client's goal was to create a prototype airline simulation capable of evaluating business feasibility, operational logistics, and customer booking workflows.

The team discussed the major components required for the system, including flight scheduling, booking functionality, employee tools, and administrative controls. During these meetings, team members reviewed the project description and identified the key features that would need to be included in the prototype. The team also discussed how the system should support profitability simulation and realistic airline operations.

Responsibilities were assigned among team members, including interface design, functionality planning, documentation preparation, and project management tracking. At the conclusion of this phase, the team established a clear project direction and identified the major modules that would be developed throughout the semester.

Mid-Semester Meetings

Focus: System design and prototype development

During the middle portion of the semester, the team met regularly to begin designing the user interface and building the system structure. These meetings focused on developing the major system pages, including login functionality, flight schedule viewing, and customer booking processes.

The team reviewed design decisions related to navigation, user roles, and page organization. Special attention was given to ensuring that different types of users like employees, and administrators could access different parts of the system. This led to the development of a role-based system that allowed the interface to display different tools depending on user type.

During this phase, the team also worked on integrating additional functionality, such as passenger booking workflows, employee flight management tools, and administrative features for monitoring operations.

Meetings often included demonstrations of newly developed features so that the team could provide feedback and suggest improvements.

Late Semester Meetings

Focus: Debugging, testing, and system improvement

During the later weeks of the semester the meetings are being focused on testing the system and identifying areas that require improvement. Team members are going to review the system workflows to ensure that each function operates correctly and that transitions between pages are smooth and reliable.

Testing includes verifying that users could log in successfully, search for flights, complete bookings, and view system information. The team is also going to test employee and administrative features to confirm that the correct tools were available to each role.

During this phase, the team is addressing minor issues related to navigation, display formatting, and usability. Improvements are made to enhance the overall appearance of the interface and ensure that system information was presented clearly.

Meetings also include the discussions about documentation progress and preparation for the final presentation. The team is reviewing screenshots, diagrams, and written materials to ensure consistency between the system design and project documentation.